

Module: Problem 5

This module provides two simple linear-search utilities that operate on a list `L`.

Both functions assume that `L` is sorted (ascending for `search`, descending-like for `newsearch`) and attempt to terminate early when the target element `e` cannot be present. They return a boolean indicating whether `e` was found.

Functions

`search(L, e)`

Description:

Iterates through `L` from the beginning. If an element equal to `e` is encountered, the function returns `True`. Because the list is presumed sorted in ascending order, the function stops and returns `False` as soon as it finds an element larger than `e`. If the end of the list is reached without a match, it returns `False`.

Parameters: - `L` (*list*): A list of items that support the `==` and `>` comparison operators (e.g., numbers, strings, booleans).

- `e` (*any*): The element being searched for; must be comparable with the items in `L`.

Returns:

`bool` - `True` if `e` is found in `L`; otherwise `False`.

Examples:

```
# Example 1 - element not present and smaller than all items
result = search([1, 2, 3], -1)          # → False

# Example 2 - element not present in a list of strings
result = search(['a', 'b', 'c'], 'test') # → False

# Example 3 - empty string not present in a single-element list
result = search(['test'], '')           # → False

# Example 4 - True is considered equal to 1 in Python, so it is found
result = search([1, 2, 3], True)        # → True
```

Edge Cases / Notes: - The function raises a `TypeError` when elements of `L` cannot be compared with `e` using `>` (e.g., mixing `str` and `int`).

- Example crash: `search(['a', 'b', 'c'], 0) → TypeError: '>' not supported between instances of 'str' and 'int'`

- Example crash: `search(['test'], 1) → TypeError: '>' not supported between instances of 'str' and 'int'` - Because the early-exit condition uses `>` it only works correctly for lists sorted in **strictly increasing** order.

`newsearch(L, e)`

Description:

Traverses `L` from both ends simultaneously. It checks the element at the current forward index `i` and the element at the mirrored backward index `size-i-1`. If either equals `e`, the function returns `True`. The function aborts early and returns `False` when it encounters a forward element that is **smaller** than `e`, assuming the list is sorted in descending order (or that larger elements appear earlier). If the loop finishes without finding `e`, it returns `False`.

Parameters: - `L` (*list*): A list whose items support the `==` and `<` comparison operators.

- `e` (*any*): The element to locate; must be comparable with the items in `L`.

Returns:

`bool` - `True` if `e` exists in `L`; otherwise `False`.

Examples:

```
# Example 1 - element not present and smaller than all items
result = newsearch([1, 2, 3], -1)          # → False

# Example 2 - element not present in a list of strings
result = newsearch(['a', 'b', 'c'], 'test') # → False

# Example 3 - empty string not present in a single-element list
result = newsearch(['test'], '')           # → False

# Example 4 - True is considered equal to 1, so it is found
result = newsearch([1, 2, 3], True)        # → True
```

Edge Cases / Notes: - A `TypeError` is raised when the `<` comparison between list items and `e` is invalid (e.g., mixing `str` and `int`).

- Crash example: `newsearch(['a', 'b', 'c'], 0) →`

`TypeError: '<' not supported between instances of 'str' and 'int'`

- Crash example: `newsearch(['test'], 1) → TypeError: '<' not supported between instances of`

`'str' and 'int'` - The early-exit logic (`L[i] < e`) only yields correct results when `L` is sorted in

non-increasing order. Using it on an ascending list may cause premature termination.